

# Oracle Analytical (Window) Functions

Ranking, offset, aggregate windows & the windowing clause · Oracle 19c

Analytic functions compute a value **across a set of rows related to the current row**, *without collapsing* them (unlike `GROUP BY`). Every row is retained, with the computed column added. Syntax: `function(args) OVER (PARTITION BY ... ORDER BY ... windowing_clause)`.

## 1. Anatomy of the OVER() Clause

| Clause                    | Purpose                                                                  |
|---------------------------|--------------------------------------------------------------------------|
| <code>PARTITION BY</code> | Split rows into groups; the function restarts per group (optional)       |
| <code>ORDER BY</code>     | Order within each partition (required for ranking/offset/running totals) |
| windowing clause          | <code>ROWS/RANGE BETWEEN ... AND ...</code> — the frame of rows used     |

## 2. Ranking Functions

| Function                  | Behaviour                            |
|---------------------------|--------------------------------------|
| <code>ROW_NUMBER()</code> | Unique sequential number, no ties    |
| <code>RANK()</code>       | Ties share a rank, then gaps (1,1,3) |
| <code>DENSE_RANK()</code> | Ties share a rank, no gaps (1,1,2)   |
| <code>NTILE(n)</code>     | Distribute rows into n buckets       |

```
SELECT ename, dept_id, sal,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY sal DESC) rn,
       RANK()        OVER (PARTITION BY dept_id ORDER BY sal DESC) rnk,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY sal DESC) drnk
FROM   emp;
```

Top-N per group — wrap in a subquery and filter on the rank:

```
SELECT * FROM (
  SELECT ename, dept_id, sal,
         ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY sal DESC) rn
  FROM   emp)
WHERE  rn <= 3;           -- top 3 salaries per dept
```

## 3. Offset Functions (LAG / LEAD)

```
SELECT ename, hiredate, sal,
       LAG(sal, 1) OVER (ORDER BY hiredate) prev_sal,
       LEAD(sal, 1) OVER (ORDER BY hiredate) next_sal,
       sal - LAG(sal) OVER (ORDER BY hiredate) diff_from_prev
FROM   emp;
```

`LAG(col, n, default)` looks n rows back; `LEAD` looks forward.

## 4. FIRST\_VALUE / LAST\_VALUE / NTH\_VALUE

```
SELECT ename, dept_id, sal,
       FIRST_VALUE(ename) OVER (PARTITION BY dept_id ORDER BY sal DESC) top_earner,
       LAST_VALUE(ename) OVER (PARTITION BY dept_id ORDER BY sal DESC
                               ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) low_earner
FROM emp;
```

`LAST_VALUE` needs an explicit frame (`ROWS BETWEEN ... UNBOUNDED FOLLOWING`); otherwise the default frame stops at the current row and returns the "wrong" value.

## 5. Aggregate Functions as Analytics (running totals)

```
SELECT ename, hiredate, sal,
       SUM(sal) OVER (ORDER BY hiredate) running_total,
       SUM(sal) OVER (PARTITION BY dept_id) dept_total,
       AVG(sal) OVER (PARTITION BY dept_id) dept_avg,
       COUNT(*) OVER (PARTITION BY dept_id) dept_headcount,
       ROUND(RATIO_TO_REPORT(sal) OVER (PARTITION BY dept_id)*100,1) pct_of_dept
FROM emp;
```

## 6. The Windowing Clause (frames)

| Frame                                                         | Rows included                                 |
|---------------------------------------------------------------|-----------------------------------------------|
| <code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code> | Start of partition → current (running total)  |
| <code>ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING</code>         | 3-row sliding window (moving average)         |
| <code>ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING</code> | Current → end of partition                    |
| <code>RANGE BETWEEN ...</code>                                | By value range of ORDER BY, not physical rows |

```
-- 3-day moving average
SELECT trx_date, amount,
       AVG(amount) OVER (ORDER BY trx_date
                        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) mov_avg_3
FROM sales;
```

## 7. Other Useful Analytics

| Function                            | Use                                         |
|-------------------------------------|---------------------------------------------|
| <code>PERCENT_RANK()</code>         | Relative rank 0..1                          |
| <code>CUME_DIST()</code>            | Cumulative distribution                     |
| <code>PERCENTILE_CONT(x)</code>     | Continuous percentile (e.g. median = 0.5)   |
| <code>LISTAGG(...) OVER(...)</code> | Concatenate values within a partition       |
| <code>FIRST / LAST (KEEP)</code>    | Aggregate tied to min/max of another column |

```
-- median salary per department
SELECT DISTINCT dept_id,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY sal)
       OVER (PARTITION BY dept_id) median_sal
FROM emp;
```

### Key points

- Analytics run **after** WHERE/GROUP BY/HAVING but **before** ORDER BY — you can't use them in a WHERE clause directly (wrap in a subquery).
- **PARTITION BY** is optional; without it the whole result set is one partition.
- Give **LAST\_VALUE** and moving windows an explicit frame or results will surprise you.
- Analytics keep all rows; use them instead of self-joins for "compare to group" problems.